

XVII. Bootstrap — Personnalisation

PERSONNALISER BOOTSTRAP EN UNE PHRASE

Changer les **COULEURS**, **typographies** et **breakpoints** de Bootstrap pour que votre site ne ressemble pas à tous les autres sites Bootstrap — soit via des variables CSS natives (simple), soit via Sass (puissant).

📘 Chapitre hors-trame séance

Ce chapitre **n'est pas traité en séance** (le Slidev J3 saute la personnalisation pour cibler 6h30 actives et libérer du temps sur le TP sommatif et l'ouverture Tailwind). Il reste dans la bible pour **lecture autonome post-cours** — réflexes utiles dès votre premier projet où Bootstrap doit s'adapter à une charte graphique imposée.

Sommaire

- **Introduction**
 - 1. Objectifs du chapitre
 - 2. En pratique — situation professionnelle
 - 3. Quand l'utiliser, quand l'éviter
- **A. Approche simple : CSS custom properties**
 - 1. Changer la couleur primaire
 - 2. Changer toute la palette
 - 3. Mode sombre automatique
- **B. Approche pro : variables Sass**
 - 1. Installation via npm
 - 2. Fichier SCSS personnalisé
 - 3. Avantages du Sass
 - 4. Exemple : importer sélectivement
- **C. Bootstrap Icons**
 - 1. Installation (CDN)
 - 2. Utilisation
 - 3. Ajuster taille et couleur
 - 4. Alternatives
- **D. Optimiser la taille du bundle**
 - 1. Utiliser le minified (défaut CDN)
 - 2. Import sélectif via Sass
 - 3. PurgeCSS / Tailwind-style
 - 4. Utiliser le bundle JS fragmenté
 - 5. Métriques cibles
- **Conclusion**
 - 1. À retenir
 - 2. Pour aller plus loin

Introduction

1. Objectifs du chapitre

À l'issue de ce chapitre, vous serez capable de :

- **Surcharger** les couleurs et typos de Bootstrap via CSS **custom properties** (variables natives).
- **Personnaliser** Bootstrap en profondeur via les variables **Sass** pour un projet pro.
- **Intégrer** les **Bootstrap Icons** dans votre site.
- **Choisir** entre CDN + CSS custom (simple) et npm + Sass (professionnel).

2. En pratique — situation professionnelle

Le client veut une charte graphique avec **son rose de marque** (#EE5BE0) au lieu du bleu Bootstrap. Sans personnalisation, toutes les pages gardent la couleur bleue par défaut. Cette leçon vous apprend à surcharger les couleurs en **3 lignes** via variables CSS ou en **10 lignes** via Sass pour personnalisation complète.

3. Quand l'utiliser, quand l'éviter

✓ Utiliser pour	✗ Éviter quand
Surcharger couleurs / typo via <code>--bs-*</code> (90 % des cas)	Empiler des <code>!important</code> partout pour écraser Bootstrap
Sass + tree-shaking pour un site de production optimisé	Compiler Sass pour un simple changement de couleur (CDN suffit)
<code>data-bs-theme="dark"</code> pour le mode sombre natif	Ré-implémenter à la main des composants déjà existants

A. Approche simple : CSS custom properties

Depuis Bootstrap 5.3, **toutes les couleurs et dimensions** sont exposées en **variables CSS natives** (`--bs-*`).

1. Changer la couleur primaire

Dans votre CSS custom, **après** l'import de Bootstrap :

```
:root {  
  --bs-primary: #EE5BE0;           /* votre rose de marque */  
  --bs-primary-rgb: 238, 91, 224; /* version RGB (requis par certaines utilities) */  
}
```

→ Tous les `.btn-primary`, `.bg-primary`, `.text-primary`, `.border-primary` deviennent **roses** automatiquement.

2. Changer toute la palette

```
:root {  
  --bs-primary: #EE5BE0;  
  --bs-secondary: #000E36;  
  --bs-success: #2ECC71;  
  --bs-danger: #E74C3C;  
  --bs-warning: #F9BA17;  
  --bs-info: #3498DB;  
  
  /* Typo */  
  --bs-body-font-family: "Inter", sans-serif;  
  --bs-body-font-size: 1rem;  
  --bs-body-line-height: 1.6;  
  
  /* Forme */  
  --bs-border-radius: 8px;  
}
```

3. Mode sombre automatique

Bootstrap 5.3 supporte le **mode sombre natif** :

```
<html data-bs-theme="dark">
```

→ Toutes les couleurs s'adaptent automatiquement. Vous pouvez même définir vos couleurs custom par thème :

```
:root {  
  --bs-primary: #EE5BE0;  
}  
[data-bs-theme="dark"] {  
  --bs-primary: #F897ED; /* plus clair pour fonder sur fond sombre */  
}
```

♦ Simplicité vs pouvoir

Cette approche **CSS custom properties** suffit pour 80 % des personnalisations. Elle ne recompile pas le CSS Bootstrap — reste rapide à mettre en place avec le CDN.

B. Approche pro : variables Sass

Pour personnaliser Bootstrap **en profondeur** (modifier les breakpoints, les échelles, la grille...), passer par les variables **Sass** avant compilation.

1. Installation via npm

```
npm install bootstrap  
npm install -D sass vite      # ou webpack, parcel...
```

2. Fichier SCSS personnalisé

```
// 1. Vos variables (avant l'import)  
$primary: #EE5BE0;  
$font-family-base: "Inter", sans-serif;  
$border-radius: 0.5rem;  
$grid-breakpoints: (  
  xs: 0,  
  sm: 640px,  
  md: 960px,  
  lg: 1200px,  
  xl: 1400px  
);  
  
// 2. Import de Bootstrap (utilise vos variables)  
@import "bootstrap/scss/bootstrap";  
  
// 3. Vos styles custom (après)  
.mon-composant {  
  // ...  
}
```

3. Avantages du Sass

- **Personnalisation complète** : breakpoints, échelles de spacing, grille.
- **Tree-shaking** : n'importer que les modules utilisés (boutons, grille, formulaires...) pour réduire la taille du CSS final.
- **Mixins réutilisables** : `@include media-breakpoint-up(md)`.

4. Exemple : importer sélectivement

```
// Variables
@import "bootstrap/scss/functions";
@import "bootstrap/scss/variables";
@import "bootstrap/scss/mixins";

// Personnalisation
$primary: #EE5BE0;

// Modules au choix (pas tous)
@import "bootstrap/scss/root";
@import "bootstrap/scss/reboot";
@import "bootstrap/scss/type";
@import "bootstrap/scss/grid";
@import "bootstrap/scss/buttons";
@import "bootstrap/scss/forms";

// On n'importe PAS les composants inutilisés (carousel, toasts...)
// → CSS compilé beaucoup plus léger (~40 kB au lieu de 150 kB)
```

C. Bootstrap Icons

Bootstrap Icons est une bibliothèque officielle de **~2 000 icônes SVG** vectorielles.

1. Installation (CDN)

Dans le `<head>` :

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.css"
      rel="stylesheet">
```

2. Utilisation

```
<i class="bi bi-house"></i> Accueil
<i class="bi bi-person-circle"></i> Mon compte
<button class="btn btn-primary">
  <i class="bi bi-cart"></i> Ajouter au panier
</button>
```

3. Ajuster taille et couleur

Les icônes héritent de la **couleur et taille du texte parent** :

```
<i class="bi bi-heart-fill text-danger fs-1"></i>  <!-- rouge, gros -->
<i class="bi bi-check-circle text-success"></i>    <!-- vert, taille normale -->
```

4. Alternatives

- **Heroicons** — style moderne, gratuit.
- **Phosphor Icons** — très complet, multi-styles.
- **Lucide** — fork moderne de Feather Icons.

D. Optimiser la taille du bundle

Bootstrap complet CSS minifié ≈ **150 kB**. Bootstrap JS minifié ≈ **80 kB**. Pour un site de production :

1. Utiliser le minified (défaut CDN)

Le CDN sert déjà `.min.css` et `.min.js`. Rien à faire.

2. Import sélectif via Sass

Voir section B.4 ci-dessus. Gain typique : 60-70 % sur le CSS.

3. PurgeCSS / Tailwind-style

Des outils comme **PurgeCSS** (souvent intégré à Vite/Webpack) **analysent votre HTML** et **suppriment toutes les classes Bootstrap non utilisées**.

```
// vite.config.js exemple
import purge from 'vite-plugin-purgecss'

export default {
  plugins: [
    purge({
      content: ['./src/**/*.html']
    })
  ]
}
```

Gain typique : **60-80 %** (on passe de 150 kB à 30-50 kB).

4. Utiliser le bundle JS fragmenté

Au lieu de `bootstrap.bundle.min.js` (tout), importer uniquement les modules JS utilisés :

```
// main.js
import 'bootstrap/js/dist/modal';
import 'bootstrap/js/dist/dropdown';
// pas d'import de carousel, toast, tooltip si non utilisés
```

5. Métriques cibles

Site production bien optimisé avec Bootstrap :

Ressource	Taille
CSS Bootstrap custom + custom	40-60 kB
JS Bootstrap (modules utilisés)	20-40 kB
Bootstrap Icons (si utilisé)	30 kB
Total	~100-130 kB

Comparé à un site moderne sans framework : ~50 kB. Trade-off vitesse/productivité vs perf pure.

Conclusion

1. À retenir

- **Personnalisation simple** : surcharger `--bs-primary`, `--bs-secondary` dans `:root` (CSS custom properties).
- **Mode sombre** : `data-bs-theme="dark"` sur `<html>`.
- **Personnalisation pro** : variables Sass **avant** l'import, tree-shaking des modules.
- **Bootstrap Icons** : ~2 000 icônes SVG, classe `bi bi-<nom>`, hérite couleur+taille du parent.
- **Optimisation bundle** : PurgeCSS pour supprimer les classes non utilisées, import JS sélectif. Cible : < 130 kB total.

2. Pour aller plus loin

- **Bootstrap — Customize**
- **Bootstrap — CSS variables**
- **Bootstrap Icons**
- **PurgeCSS**